MLOps Laboratory

Experiment No. 07: Implement CI/CD Pipeline using GitHub Actions

1. Student Details

- Roll No.: 64
- Name: Manthan Butala
- Division / Batch: CSAI-A
- Practical No.: 07

2. Title of Practical

Implement CI/CD Pipeline using GitHub Actions — Automating build, test, and deployment stages of an ML pipeline through a version-controlled, event-triggered workflow.

3. Objectives

- To understand the concept of Continuous Integration and Continuous Deployment (CI/CD) in the context of MLOps.
- To design and configure a GitHub Actions workflow that automatically builds, lints, and tests code on every push/pull request.
- To observe how a CI/CD pipeline detects failures (e.g., failing tests) and confirms successful runs, ensuring reliable and reproducible integration of code changes.

4. Problem Statement / Practical Description

In an ML project, code changes made by multiple contributors must be validated automatically before being merged or deployed, to avoid breaking the pipeline. Manually running tests and checks for every change is slow and error-prone. This practical addresses this problem by implementing a CI/CD pipeline using GitHub Actions, which automatically triggers a workflow on every push or pull request to the repository. The workflow checks out the code, sets up the Python environment, installs dependencies, runs static code checks (linting) and unit tests, and reports the result — automating the integration stage of the ML development lifecycle.

5. Tools & Technologies Used

- GitHub (repository hosting and version control)
- GitHub Actions (CI/CD automation, cloud-hosted Ubuntu runner)
- Git
- Python 3.12
- pytest (unit testing)
- flake8 (linting / static code analysis)
- Visual Studio Code (for editing project and workflow files)

6. Dataset Details

This practical does not use an external dataset. It operates on a small sample Python module and its accompanying unit tests, used purely to demonstrate the CI/CD automation:

- Dataset name: Not applicable — sample source code and test files only

- Dataset source/link: Not applicable
- Number of records/features: Not applicable — a single utility module (calc.py) with 2 functions (add, divide) and 3 corresponding test cases
- Brief description of important attributes: The add() and divide() functions represent the ML utility code under test; the divide() function additionally validates error handling (division by zero) to demonstrate a meaningful automated test case

7. Methodology / Implementation

Step 1: Create the GitHub Repository and Clone Locally

A new repository is created on GitHub and cloned to the local machine to begin building the project structure.

```
git clone https://github.com/bhargavi-borkhade/mlops-cicd-demo.git
cd mlops-cicd-demo
```

Expected Output:

```
Cloning into 'mlops-cicd-demo'...
done.
```

Step 2: Create the Project Structure

A minimal Python project is set up with a source module, a test file, and a requirements file, so the workflow has something concrete to build, lint, and test.

```
mlops-cicd-demo/
├── src/
│   └── calc.py
├── tests/
│   └── test_calc.py
├── requirements.txt
└── .github/
    └── workflows/
        └── ci.yml
```

Step 3: Write the Source Module (src/calc.py)

```
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Step 4: Write Unit Tests (tests/test_calc.py)

```
import pytest
from src.calc import add, divide

def test_add():
    assert add(2, 3) == 5

def test_divide():
    assert divide(10, 2) == 5

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)
```

Step 5: Add requirements.txt

```
pytest==8.2.0
flake8==7.0.0
```

Step 6: Create the GitHub Actions Workflow File (.github/workflows/ci.yml)

The workflow is triggered on every push and pull request to the main branch. It runs on an Ubuntu runner, sets up Python, installs dependencies, runs flake8 for linting, and runs pytest for unit tests.

```
name: CI Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Lint with flake8
        run: flake8 src tests --max-line-length=100

      - name: Run unit tests with pytest
        run: pytest -v
```

Expected Output: The workflow file is saved without syntax errors inside .github/workflows/.

Step 7: Commit and Push to Trigger the Pipeline

Pushing the project (including the workflow file) to the main branch automatically triggers the CI pipeline defined above.

```
git add .
git commit -m "Add source module, tests, and CI workflow"
git push origin main
```

Expected Output:

```
Enumerating objects...
To github.com:bhargavi-borkhade/mlops-cicd-demo.git
   main -> main
```

Step 8: Observe the Workflow Run in the Actions Tab

The "Actions" tab of the GitHub repository is opened. The triggered "CI Pipeline" run is inspected, expanding each step (Checkout, Set up Python, Install dependencies, Lint, Run tests) to confirm execution order and status.

Expected Output: All steps complete with a green checkmark; the job build-and-test shows overall status SUCCESS.

Step 9: Introduce a Failing Test to Verify Failure Detection

To validate that the pipeline correctly detects broken code, the assertion in test_add() is deliberately changed to an incorrect expected value, and the change is pushed.

```
def test_add():
    assert add(2, 3) == 6   # intentionally incorrect
```

Expected Output: The GitHub Actions run fails at the "Run unit tests with pytest" step; the job and the commit are marked with a red cross (FAILURE), and the log shows an AssertionError for test_add.

Step 10: Fix the Test and Confirm Pipeline Recovery

The assertion is corrected back to the valid expected value and pushed again, re-triggering the workflow.

```
def test_add():
    assert add(2, 3) == 5    # corrected
```

Expected Output: The new workflow run completes successfully; all checks pass and the commit is marked with a green checkmark (SUCCESS), confirming the pipeline correctly gates code changes on passing tests.

8. Source Code

The complete source code — src/calc.py, tests/test_calc.py, requirements.txt, and the GitHub Actions workflow .github/workflows/ci.yml — is provided in Steps 3 to 6 above under Methodology / Implementation.

9. Output / Results

Observation Table

Workflow Run	Trigger	Steps Executed	Status
Initial push (correct code)	git push to main	Checkout, Setup Python, Install deps, Lint, Test	Success
Push with broken test_add()	git push to main	Checkout, Setup Python, Install deps, Lint, Test	Failure (AssertionError)
Push with corrected test	git push to main	Checkout, Setup Python, Install deps, Lint, Test	Success

Screenshots confirming the GitHub Actions run list, the expanded step-by-step logs for the successful run, the red-cross failed run with the AssertionError log, and the subsequent green-check recovered run are attached as per the practical execution.

10. Result Analysis

The results confirm that the GitHub Actions workflow correctly automates the integration pipeline. On the initial push, all five steps (checkout, environment setup, dependency installation, linting, and testing) executed in sequence and completed successfully, verifying that the workflow YAML was syntactically valid and correctly ordered. When a faulty assertion was introduced in test_add(), the pipeline correctly halted at the testing step and reported a failure, demonstrating that GitHub Actions reliably gates the codebase against regressions without requiring any manual verification. After the fix was pushed, the pipeline re-ran automatically and returned to a SUCCESS state, confirming that the CI pipeline is re-triggered on every commit and accurately reflects the current health of the codebase.

11. Conclusion

A CI/CD pipeline was successfully implemented using GitHub Actions to automate the build, lint, and test stages of an ML codebase. The workflow was shown to trigger automatically on every push, execute all steps in the defined order, and correctly distinguish between passing and failing code changes. This practical demonstrated how CI/CD removes manual verification overhead and enforces code quality and reliability continuously, reinforcing the MLOps principle — already seen with Git/DVC for data and Airflow for pipeline orchestration — that reproducibility and automation must extend across the entire ML development lifecycle, including code integration and delivery.